

Module-II

I/O devices, and I/O subsystems

I/O devices are peripheral devices that are connected to a computer to allow communication between the computer and the outside world. The I/O subsystem is a part of a computer that manages all the input and output operations of the computer system.

I/O devices are very important in the computer systems. They provide users the means of interacting with the system. So there is a separate I/O system devoted to handling the I/O devices.

The different Components of the I/O systems are –

I/O Hardware

There are many I/O devices handled by the operating system such as mouse, keyboard, disk drive etc. There are different device drivers that can be connected to the operating system to handle a specific device. The device controller is an interface between the device and the device driver.

A diagram to represent this is –

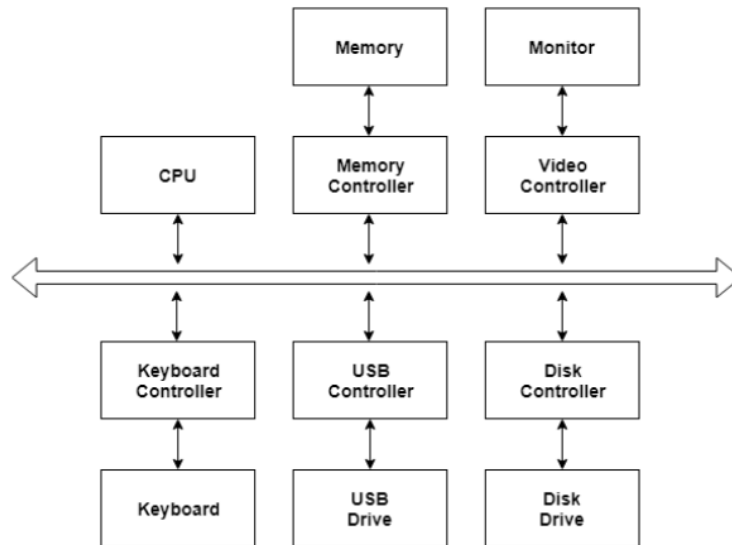
I/O devices are very important in the computer systems. They provide users the means of interacting with the system. So there is a separate I/O system devoted to handling the I/O devices.

The different Components of the I/O systems are –

I/O Hardware

There are many I/O devices handled by the operating system such as mouse, keyboard, disk drive etc. There are different device drivers that can be connected to the operating system to handle a specific device. The device controller is an interface between the device and the device driver.

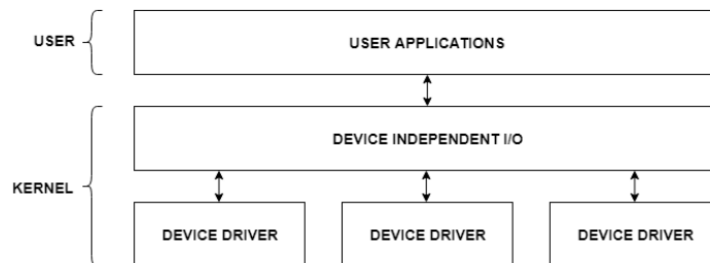
A diagram to represent this is –



I/O Application Interface

The user applications can access all the I/O devices using the device drivers, which are device specific codes. The application layer sees a common interface for all the devices.

This is illustrated using the below image –



Most of the devices are either block I/O or character I/O devices. Block devices are accessed one block at a time whereas character devices are accessed one character at a time.

I/O Software

The I/O software contains the user level libraries and the kernel modules. The libraries provide the interface to the user program to perform input and output. The kernel modules provides the device drivers that interact with the device controllers.

The I/O software should be device independent so that the programs can be used for any I/O device without specifying it in advance. For example - A program that reads a file should be

able to read the file on a hard disk, floppy disk, CD-ROM etc. without having to change the program each time.

I/O buffering and its Various Techniques

A buffer is a memory area that stores data being transferred between two devices or between a device and an application. I/O buffering is a technique used in computers to manage data transfer between the computer's memory and input/output devices (like hard drives, printers, or network devices). It helps make data transfer more efficient by temporarily storing data in a buffer, which is a reserved area of memory. This allows the CPU and I/O devices to work at their speeds without having to wait for each other, improving overall system performance.

I/O Buffering

I/O buffering is a technique used in computer systems to improve the efficiency of input and output (I/O) operations. It involves the temporary storage of data in a buffer, which is a reserved area of memory, to reduce the number of I/O operations and manage the flow of data between fast and slow devices or processes.

Uses of I/O Buffering

- Buffering is done to deal effectively with a speed mismatch between the producer and consumer of the data stream.
- A buffer is produced in the main memory to heap up the bytes received from the modem.
- After receiving the data in the buffer, the data gets transferred to disk from the buffer in a single operation.
- This process of data transfer is not instantaneous, therefore the modem needs another buffer to store additional incoming data.
- When the first buffer is filled, then it is requested to transfer the data to disk.
- The modem then starts filling the additional incoming data in the second buffer while the data in the first buffer gets transferred to the disk.
- When both buffers complete their tasks, then the modem switches back to the first buffer while the data from the second buffer gets transferred to the disk.
- The use of two buffers disintegrates the producer and the consumer of the data, thus minimizing the time requirements between them.
- Buffering also provides variations for devices that have different data transfer sizes.

Types of I/O Buffering Techniques

1. Single Buffer

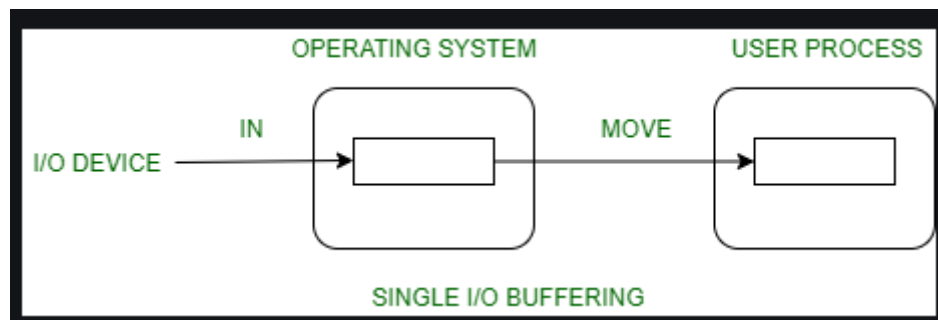
Using one buffer to store data temporarily. A buffer is provided by the operating system to the system portion of the main memory.

Block Oriented Device

- The system buffer takes the input.
- After taking the input, the block gets transferred to the user space by the process and then the process requests for another block.
- Two blocks work simultaneously, when one block of data is processed by the user process, the next block is being read in.
- OS can swap the processes.
- OS can record the data of the system buffer to user processes.

Stream Oriented Device

- Line- at a time operation is used for scroll-made terminals. The user inputs one line at a time, with a carriage return signalling at the end of a line.
- Byte-at-a-time operation is used on forms mode, terminals when each keystroke is significant.



2. Double Buffer

In this technique the operating system uses two buffers to allow continuous data transfer between two process and two devices.

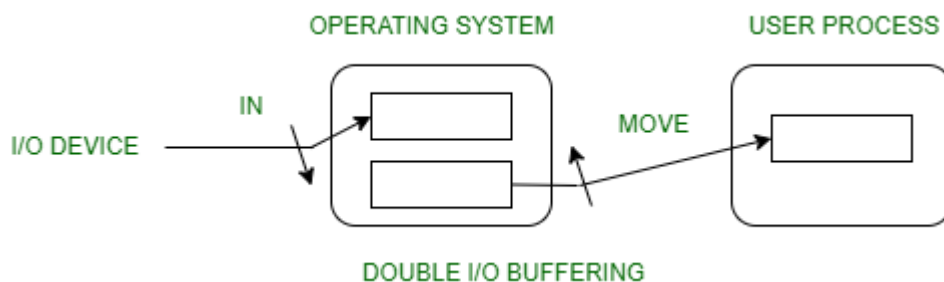
Block Oriented

- There are two buffers in the system.
- One buffer is used by the driver or controller to store data while waiting for it to be taken by higher level of the hierarchy.
- Other buffer is used to store data from the lower level module.

- Double buffering is also known as buffer swapping.
- A major disadvantage of double buffering is that the complexity of the process get increased.
- If the process performs rapid bursts of I/O, then using double buffering may be deficient.

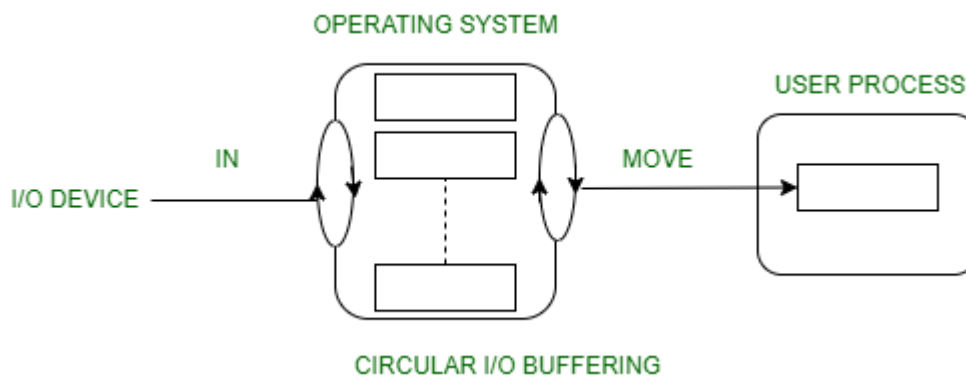
Stream Oriented

- Line- at a time I/O, the user process need not be suspended for input or output, unless process runs ahead of the double buffer.
- Byte- at a time operations, double buffer offers no advantage over a single buffer of twice the length.



3. Circular Buffer

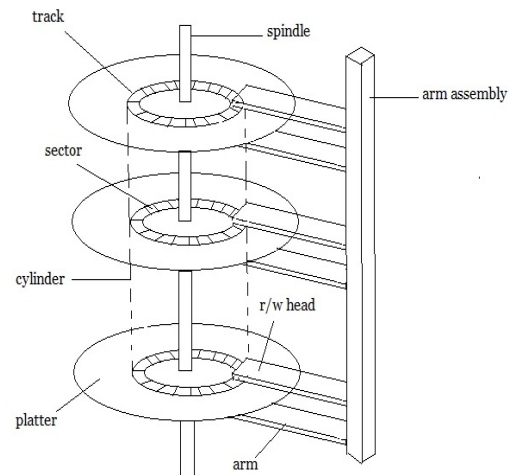
- In this technique the operating system uses a circular buffer to manage continuous data streams efficiently.
- When more than two buffers are used, the collection of buffers is itself referred to as a circular buffer.
- In this, the data do not directly passed from the producer to the consumer because the data would change due to overwriting of buffers before they had been consumed.
- The producer can only fill up to buffer $i-1$ while data in buffer i is waiting to be consumed.



Disk Storage

Hard disk is the collection of multiple platters and every platter have a different track and every track have a different sectors. Data store in a sector sectors.

Patter-surface-tack-sector



Disk Pack Structure

As we know, a process needs two type of time, CPU time and IO time. For I/O, it requests the Operating system to access the disk.

However, the operating system must be fare enough to satisfy each request and at the same time, operating system must maintain the efficiency and speed of process execution.

The technique that operating system uses to determine the request which is to be satisfied next is called disk scheduling.

Let's discuss some important terms related to disk scheduling.

Seek Time

Seek time is the time taken in locating the disk arm to a specified track where the read/write request will be satisfied.

Rotational Latency

It is the time taken by the desired sector to rotate itself to the position from where it can access the R/W heads.

Transfer Time

It is the time taken to transfer the data.

Disk Access Time

Disk access time is given as,

Disk Access Time = Rotational Latency + Seek Time + Transfer Time

Disk Response Time

It is the average of time spent by each request waiting for the IO operation.

Purpose of Disk Scheduling

The main purpose of disk scheduling algorithm is to select a disk request from the queue of IO requests and decide the schedule when this request will be processed.

Goal of Disk Scheduling Algorithm

- Fairness
- High throughput
- Minimal traveling head time

Disk Scheduling Algorithms

The list of various disks scheduling algorithm is given below. Each algorithm is carrying some advantages and disadvantages. The limitation of each algorithm leads to the evolution of a new algorithm.

- FCFS scheduling algorithm
- SSTF (shortest seek time first) algorithm
- SCAN scheduling
- C-SCAN scheduling
- LOOK Scheduling
- C-LOOK scheduling

FCFS Scheduling Algorithm

It is the simplest Disk Scheduling algorithm. It services the IO requests in the order in which they arrive. There is no starvation in this algorithm, every request is serviced.

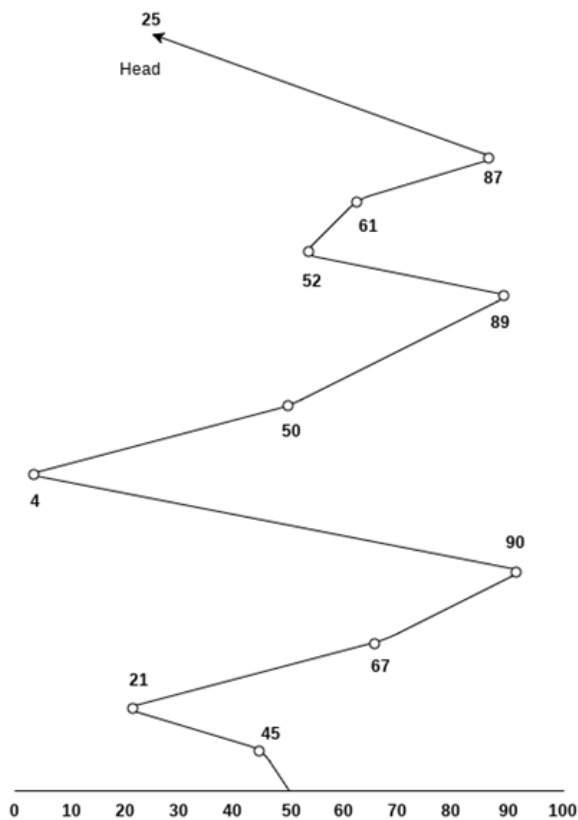
Disadvantages

- The scheme does not optimize the seek time.
- The request may come from different processes therefore there is the possibility of inappropriate movement of the head.

Example

Consider the following disk request sequence for a disk with 100 tracks 45, 21, 67, 90, 4, 50, 89, 52, 61, 87, 25

Head pointer starting at 50 and moving in left direction. Find the number of head movements in cylinders using FCFS scheduling.



Number of cylinders moved by the head

$$= (50-45)+(45-21)+(67-21)+(90-67)+(90-4)+(50-4)+(89-50)+(61-52)+(87-61)+(87-25)$$

$$= 5 + 24 + 46 + 23 + 86 + 46 + 49 + 9 + 26 + 62$$

$$= 376$$

SSTF Scheduling Algorithm

Shortest seek time first (SSTF) algorithm selects the disk I/O request which requires the least disk arm movement from its current position regardless of the direction. It reduces the total seek time as compared to FCFS.

It allows the head to move to the closest track in the service queue.

Disadvantages

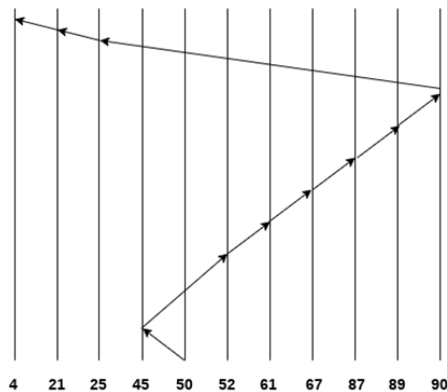
- It may cause starvation for some requests.
- Switching direction on the frequent basis slows the working of algorithm.
- It is not the most optimal algorithm.

Example

Consider the following disk request sequence for a disk with 100 tracks

45, 21, 67, 90, 4, 89, 52, 61, 87, 25

Solution:



Number of cylinders = $5 + 7 + 9 + 6 + 20 + 2 + 1 + 65 + 4 + 17 = 136$

SCAN and C-SCAN algorithm

Scan Algorithm

It is also called as Elevator Algorithm. In this algorithm, the disk arm moves into a particular direction till the end, satisfying all the requests coming in its path, and then it turns back and moves in the reverse direction satisfying requests coming in its path.

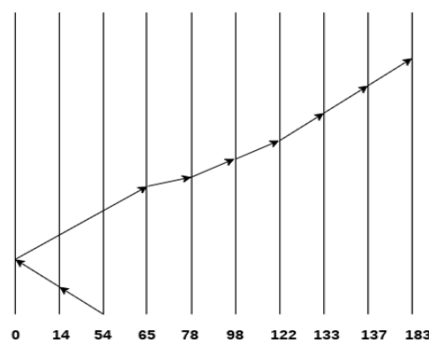
It works in the way an elevator works, elevator moves in a direction completely till the last floor of that direction and then turns back.

Example

Consider the following disk request sequence for a disk with 100 tracks

98, 137, 122, 183, 14, 133, 65, 78

Head pointer starting at 54 and moving in left direction. Find the number of head movements in cylinders using SCAN scheduling.



Number of Cylinders = $40 + 14 + 65 + 13 + 20 + 24 + 11 + 4 + 46 = 237$

C-SCAN algorithm

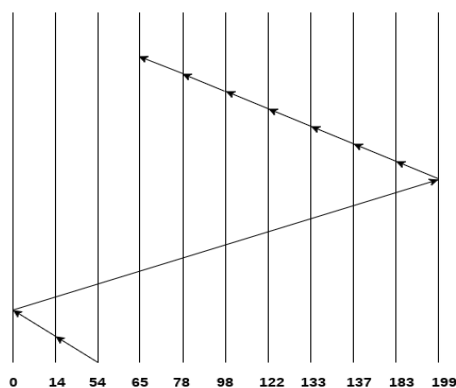
In C-SCAN algorithm, the arm of the disk moves in a particular direction servicing requests until it reaches the last cylinder, then it jumps to the last cylinder of the opposite direction without servicing any request then it turns back and start moving in that direction servicing the remaining requests.

Example

Consider the following disk request sequence for a disk with 100 tracks

98, 137, 122, 183, 14, 133, 65, 78

Head pointer starting at 54 and moving in left direction. Find the number of head movements in cylinders using C-SCAN scheduling.



No. of cylinders crossed = $40 + 14 + 199 + 16 + 46 + 4 + 11 + 24 + 20 + 13 = 387$

Look Scheduling

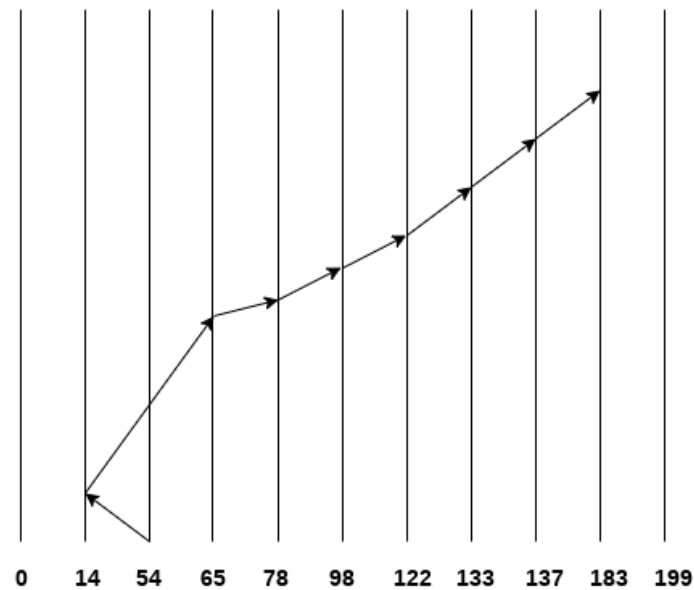
It is like SCAN scheduling Algorithm to some extent except the difference that, in this scheduling algorithm, the arm of the disk stops moving inwards (or outwards) when no more request in that direction exists. This algorithm tries to overcome the overhead of SCAN algorithm which forces disk arm to move in one direction till the end regardless of knowing if any request exists in the direction or not.

Example

Consider the following disk request sequence for a disk with 100 tracks

98, 137, 122, 183, 14, 133, 65, 78

Head pointer starting at 54 and moving in left direction. Find the number of head movements in cylinders using LOOK scheduling.



Number of cylinders crossed = $40 + 51 + 13 + 20 + 24 + 11 + 4 + 46 = 209$

C Look Scheduling

C Look Algorithm is similar to C-SCAN algorithm to some extent. In this algorithm, the arm of the disk moves outwards servicing requests until it reaches the highest request cylinder, then it jumps to the lowest request cylinder without servicing any request then it again start moving outwards servicing the remaining requests.

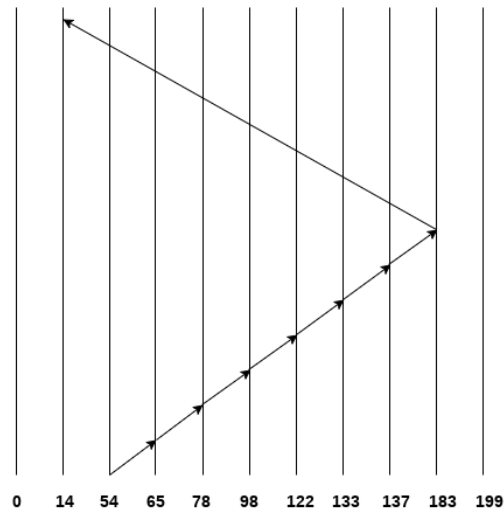
It is different from C SCAN algorithm in the sense that, C SCAN force the disk arm to move till the last cylinder regardless of knowing whether any request is to be serviced on that cylinder or not.

Example

Consider the following disk request sequence for a disk with 100 tracks

98, 137, 122, 183, 14, 133, 65, 78

Head pointer starting at 54 and moving in left direction. Find the number of head movements in cylinders using C LOOK scheduling.



Number of cylinders crossed = $11 + 13 + 20 + 24 + 11 + 4 + 46 + 169 = 298$

Numerical on Disk Scheduling Algorithms

Q. Consider a disk with 200 tracks and the queue has random requests from different processes in the order:

55, 58, 39, 18, 90, 160, 150, 38, 184

Initially arm is at 100. Find the Average Seek length using FIFO, SSTF, SCAN and C-SCAN algorithm.

Solution

Using FCFS		Using SSTF		Using SCAN		Using C - SCAN	
Next request to be serviced	No. of cylinders travelled	Next request to be serviced	No. of cylinders travelled	Next request to be serviced	No. of cylinders travelled	Next request to be serviced	No. of cylinders travelled
55	45	90	10	150	50	150	50
58	3	58	32	160	10	160	10
39	19	55	3	184	24	184	24
18	21	39	16	90	94	18	166
90	72	38	1	58	32	38	20
160	70	18	20	55	3	39	1
150	10	150	132	39	16	55	16
38	112	160	10	38	1	58	3
184	146	184	24	18	20	90	32
Average Seek Length	$498 / 9 = 55.3$	Average Seek Length	27.5	Average Seek Length	27.8	Average Seek Length	35.6

Use Case: Suitable for mission-critical applications where data availability and fault tolerance are paramount, such as in large-scale data storage environments. RAID works

RAID, or Redundant Array of Independent Disks, operates by aggregating multiple physical disk drives into a single cohesive unit to achieve data redundancy, improved performance, or both. Here's an in-depth look at how RAID functions:

Core Mechanisms of RAID:

Data Striping:

Concept: Data striping involves splitting data into small, manageable chunks (stripes) and distributing them across multiple disks. Each stripe is written to a different disk in the array.

Functionality: This method increases data read/write speeds because multiple disks can be accessed concurrently, allowing parallel I/O operations.

2. Data Mirroring:

Concept: Data mirroring duplicates identical copies of data on two or more disks.

Functionality: This provides high data availability and fault tolerance. In the event of a disk failure, the system can switch to the mirrored disk without data loss or downtime.

3. Parity Information:

Concept: Parity involves adding extra data (parity bits) to the stored information to detect and correct errors.

Functionality: Parity bits are calculated and written alongside the data. In the event of a disk failure, the parity information can be used to reconstruct the lost data. This method balances redundancy and storage efficiency, especially in RAID 5 and RAID 6 configurations.

RAID Levels and Their Operations

1. RAID 0 (Striping):

Operation: Data is striped across all disks in the array with no redundancy. This level offers maximum performance but no fault tolerance. If one disk fails, all data is lost.

Use Case: Suitable for non-critical systems where high-speed data access is prioritized, such as in video editing workstations.

2. RAID 1 (Mirroring):

Operation: Data is mirrored across two or more disks, providing redundancy. All data is written identically to each disk in the array.

Use Case: Ideal for systems requiring high availability and data protection, such as in financial and transactional databases.

3. RAID 2 (Bit-level Striping with Hamming Code Parity):

Operation: Uses bit-level striping and hamming code for error correction. This setup is complex and not commonly used in modern systems.

Use Case: Historically used in environments where high error correction was critical, though largely obsolete today.

4. RAID 3 (Byte-level Striping with Dedicated Parity):

Operation: Employs byte-level striping with a single dedicated parity disk. This setup enhances performance for large sequential data transfers but can be a bottleneck for random I/O operations.

Use Case: Suitable for applications with large, continuous data streams, such as video streaming servers.

5. RAID 4 (Block-level Striping with Dedicated Parity):

Operation: Implements block-level striping with a dedicated parity disk. While it improves read performance, write operations may suffer due to the parity disk becoming a bottleneck.

Use Case: Useful in environments where read operations significantly outnumber writes.

6. RAID 5 (Block-level Striping with Distributed Parity):

Operation: Distributes both data and parity information across all disks. This setup balances performance, fault tolerance, and storage efficiency. RAID 5 can recover from a single disk failure.

Use Case: Common in enterprise systems and data centres where a mix of performance and redundancy is needed.

7. RAID 6 (Block-level Striping with Dual Distributed Parity):

Operation: Extends RAID 5 by adding an additional parity block, enabling the array to withstand two simultaneous disk failures.

File System

File Allocation Methods

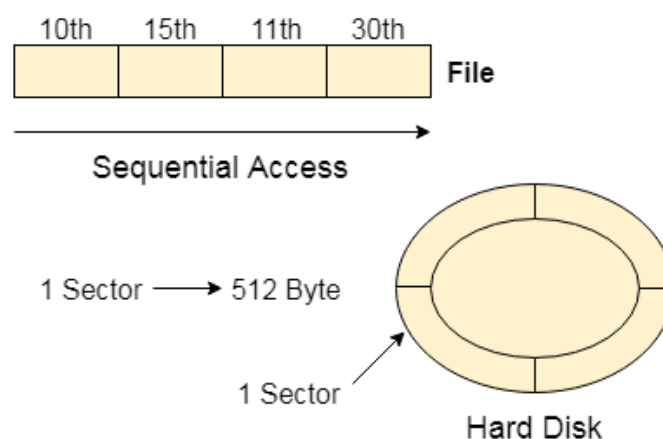
□ There are three typical file space allocation methods:

- ✦ Contiguous Allocation
- ✦ Linked Allocation
- ✦ Indexed Allocation

File Access Methods

Let's look at various ways to access files stored in secondary memory.

Sequential Access



Most of the operating systems access the file sequentially. In other words, we can say that most of the files need to be accessed sequentially by the operating system.

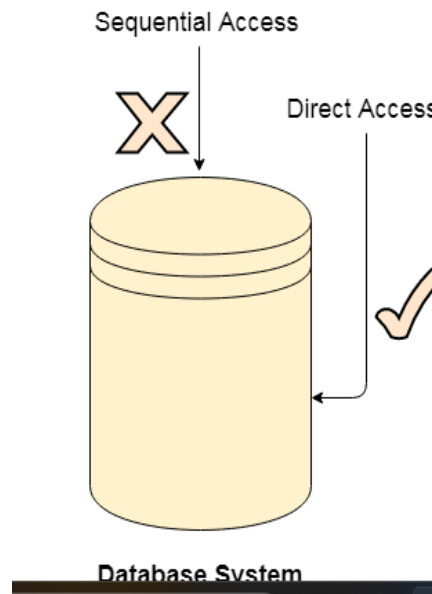
In sequential access, the OS read the file word by word. A pointer is maintained which initially points to the base address of the file. If the user wants to read first word of the file then the pointer provides that word to the user and increases its value by 1 word. This process continues till the end of the file.

Direct Access

The Direct Access is mostly required in the case of database systems. In most of the cases, we need filtered information from the database. The sequential access can be very slow and inefficient in such cases.

Suppose every block of the storage stores 4 records and we know that the record we needed is stored in 10th block. In that case, the sequential access will not be implemented because it will traverse all the blocks in order to access the needed record.

Modern word systems do provide the concept of direct access and indexed access but the most used method is sequential access due to the fact that most of the files such as text files, audio files, video files, etc need to be sequentially accessed.



Indexed Access

If a file can be sorted on any of the filed then an index can be assigned to a group of certain records. However, A particular record can be accessed by its index. The index is nothing but the address of a record in the file.

In index accessing, searching in a large database became very quick and easy but we need to have some extra space in the memory to store the index value.

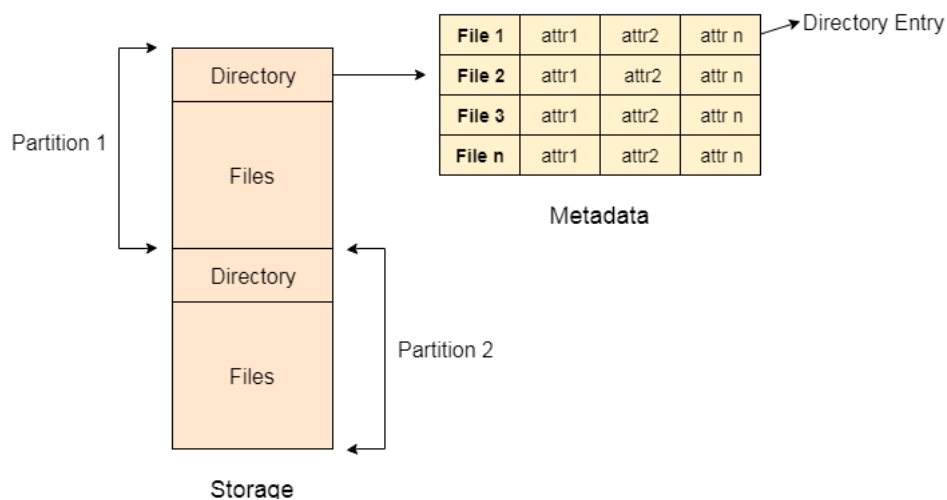
Directory Structure in OS (Operating System)

What is a directory?

Directory can be defined as the listing of the related files on the disk. The directory may store some or the entire file attributes.

To get the benefit of different file systems on the different operating systems, A hard disk can be divided into the number of partitions of different sizes. The partitions are also called volumes or mini disks.

Each partition must have at least one directory in which, all the files of the partition can be listed. A directory entry is maintained for each file in the directory which stores all the information related to that file.



A directory can be viewed as a file which contains the Meta data of the bunch of files.

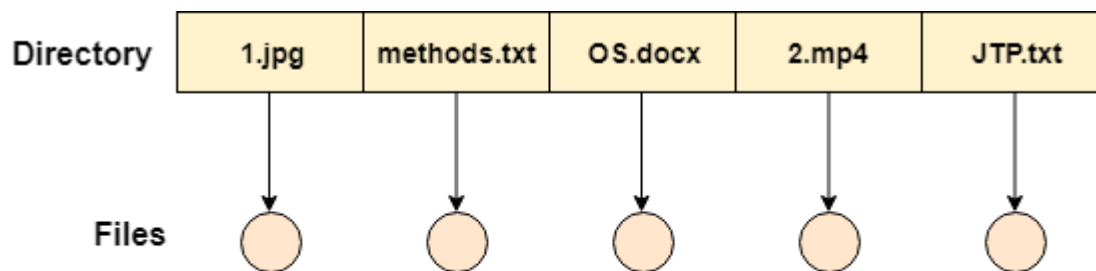
Every Directory supports a number of common operations on the file:

1. File Creation
2. Search for the file
3. File deletion
4. Renaming the file

5. Traversing Files
6. Listing of files

Single Level Directory

The simplest method is to have one big list of all the files on the disk. The entire system will contain only one directory which is supposed to mention all the files present in the file system. The directory contains one entry per each file present on the file system.



Single Level Directory

This type of directories can be used for a simple system.

Advantages

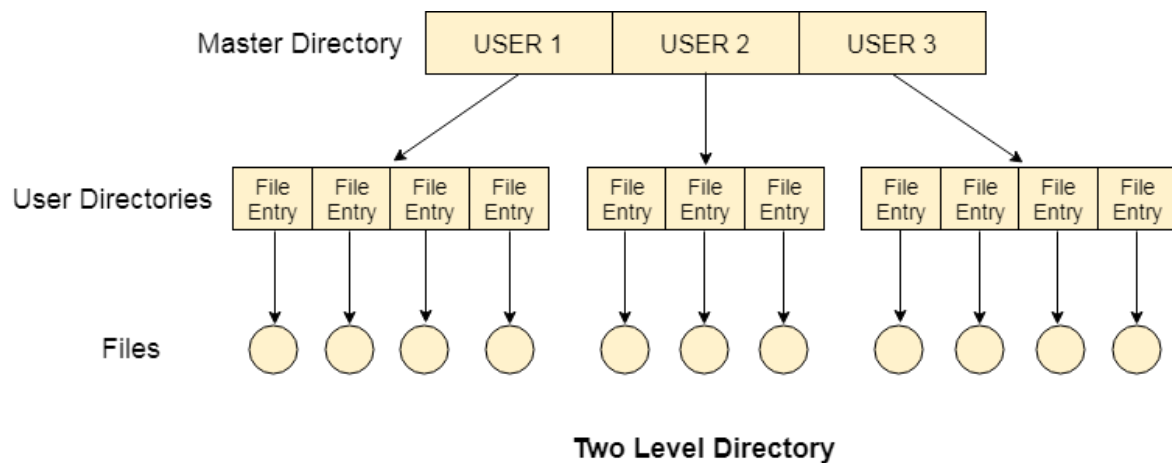
1. Implementation is very simple.
2. If the sizes of the files are very small then the searching becomes faster.
3. File creation, searching, deletion is very simple since we have only one directory.

Disadvantages

1. We cannot have two files with the same name.
2. The directory may be very big therefore searching for a file may take so much time.
3. Protection cannot be implemented for multiple users.
4. There are no ways to group same kind of files.
5. Choosing the unique name for every file is a bit complex and limits the number of files in the system construct the file name. Because most of the Operating System limits the number of characters used to

Two Level Directory

In two level directory systems, we can create a separate directory for each user. There is one master directory which contains separate directories dedicated to each user. For each user, there is a different directory present at the second level, containing group of user's file. The system doesn't let a user to enter in the other user's directory without permission.



Characteristics of two level directory system

1. Each files has a path name as */User-name/directory-name/*
2. Different users can have the same file name.
3. Searching becomes more efficient as only one user's list needs to be traversed.
4. The same kind of files cannot be grouped into a single directory for a particular user.

Every Operating System maintains a variable as **PWD** which contains the present directory name (present user name) so that the searching can be done appropriately.

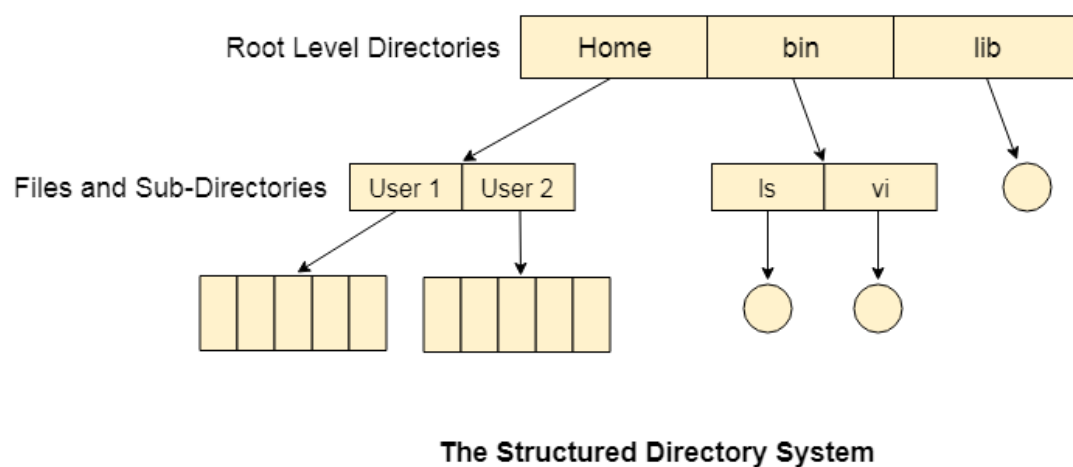
Tree Structured Directory

In Tree structured directory system, any directory entry can either be a file or sub directory. Tree structured directory system overcomes the drawbacks of two level directory system. The similar kind of files can now be grouped in one directory.

Each user has its own directory and it cannot enter in the other user's directory. However, the user has the permission to read the root's data but he cannot write or modify this. Only administrator of the system has the complete access of root directory.

Searching is more efficient in this directory structure. The concept of current working directory is used. A file can be accessed by two types of path, either relative or absolute.

Absolute path is the path of the file with respect to the root directory of the system while relative path is the path with respect to the current working directory of the system. In tree structured directory systems, the user is given the privilege to create the files as well as directories.



Permissions on the file and directory

A tree structured directory system may consist of various levels therefore there is a set of permissions assigned to each file and directory.

The permissions are **R W X** which are regarding reading, writing and the execution of the files or directory. The permissions are assigned to three types of users: owner, group and others.

There is a identification bit which differentiate between directory and file. For a directory, it is **d** and for a file, it is dot (**.**)

The following snapshot shows the permissions assigned to a file in a Linux based system. Initial bit **d** represents that it is a directory.

Acyclic-Graph Structured Directories

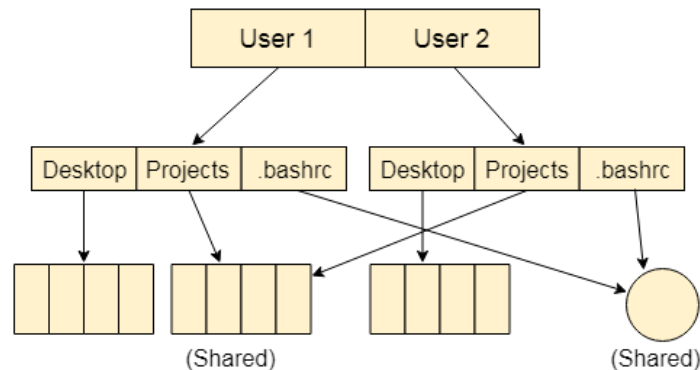
The tree structured directory system doesn't allow the same file to exist in multiple directories therefore sharing is major concern in tree structured directory system. We can provide sharing by making the directory an acyclic graph. In this system, two or more directory entry can point to the same file or sub directory. That file or sub directory is shared between the two directory entries.

These kinds of directory graphs can be made using links or aliases. We can have multiple paths for a same file. Links can either be symbolic (logical) or hard link (physical).

If a file gets deleted in acyclic graph structured directory system, then

1. In the case of soft link, the file just gets deleted and we are left with a dangling pointer.

2. In the case of hard link, the actual file will be deleted only if all the references to it gets deleted.



Acyclic-Graph Structured Directory System

File Systems

File system is the part of the operating system which is responsible for file management. It provides a mechanism to store the data and access to the file contents including data and programs. Some Operating systems treats everything as a file for example Ubuntu.

The File system takes care of the following issues

- **File Structure**

We have seen various data structures in which the file can be stored. The task of the file system is to maintain an optimal file structure.

- **Recovering Free space**

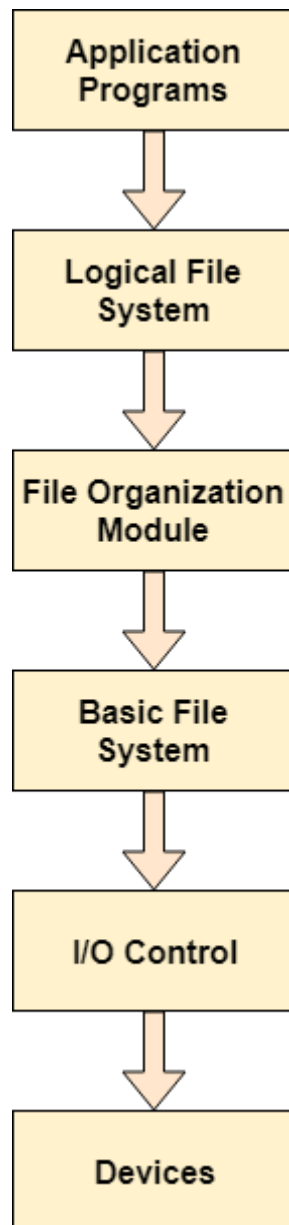
Whenever a file gets deleted from the hard disk, there is a free space created in the disk. There can be many such spaces which need to be recovered in order to reallocate them to other files.

- **disk space assignment to the files**

The major concern about the file is deciding where to store the files on the hard disk. There are various disks scheduling algorithm which will be covered later in this tutorial.

- **tracking data location**

A File may or may not be stored within only one block. It can be stored in the non-contiguous blocks on the disk. We need to keep track of all the blocks on which the part of the files reside.



- When an application program asks for a file, the first request is directed to the logical file system. The logical file system contains the Meta data of the file and directory structure. If the application program doesn't have the required permissions of the file then this layer will throw an error. Logical file systems also verify the path to the file.
- Generally, files are divided into various logical blocks. Files are to be stored in the hard disk and to be retrieved from the hard disk. Hard disk is divided into various tracks and sectors. Therefore, in order to store and

retrieve the files, the logical blocks need to be mapped to physical blocks. This mapping is done by File organization module. It is also responsible for free space management.

- Once File organization module decided which physical block the application program needs, it passes this information to basic file system. The basic file system is responsible for issuing the commands to I/O control in order to fetch those blocks.
- I/O controls contain the codes by using which it can access hard disk. These codes are known as device drivers. I/O controls are also responsible for handling interrupts.